

Extra credit 1. The webhook receiver

Not Saturday. Do not skip Module 2 for this.

New trigger. Same exits. Same harness. The graph does not change.

One FastAPI server GitHub can POST to. Extra credit 2 and 5 both need it.

Filled answer: `solutions/extra_credit/s_ext_1_webhook/`

Spillwave Solutions | spillwave.com

What you will build

A receiver that **starts** one poll. It does not grade the ticket.

	ROUTE	JOB
GET	/health	backend name and sol1 path
POST	/github-webhook	verify HMAC, reply 202, spawn task run

It shells out to `solutions/sol1_enhancer`. It does **not** import it.

Stub: `labs/extra-credit/ext_1_webhook/webhook_server.py`

Why this exists

`task poll-forever` is a seminar lie. Close the laptop and polling stops.

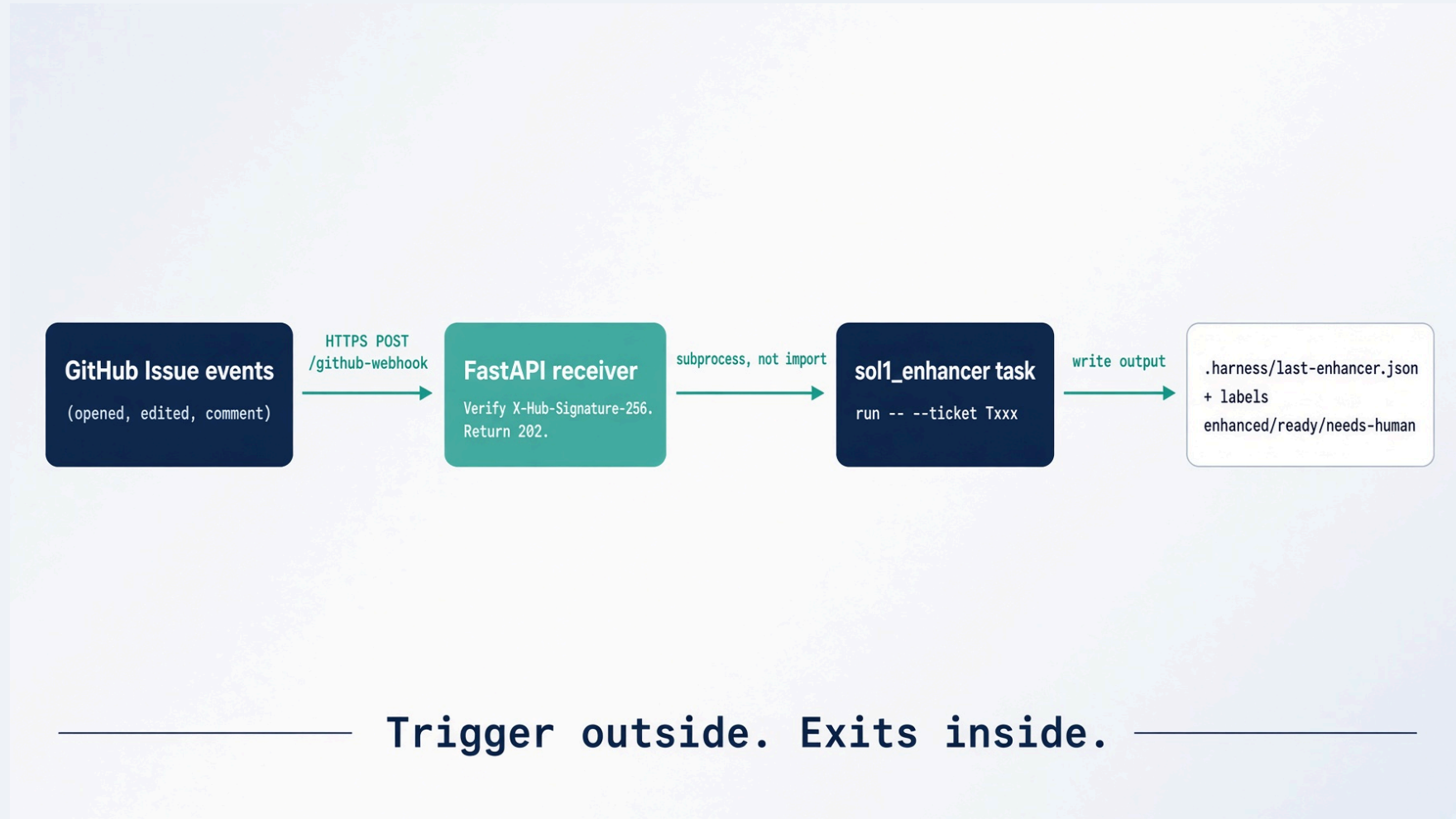
Production is an event. GitHub already knows when a ticket changed.

The trigger moves out of the loop. The exits stay in it. A webhook starts the run. It never decides when to stop.

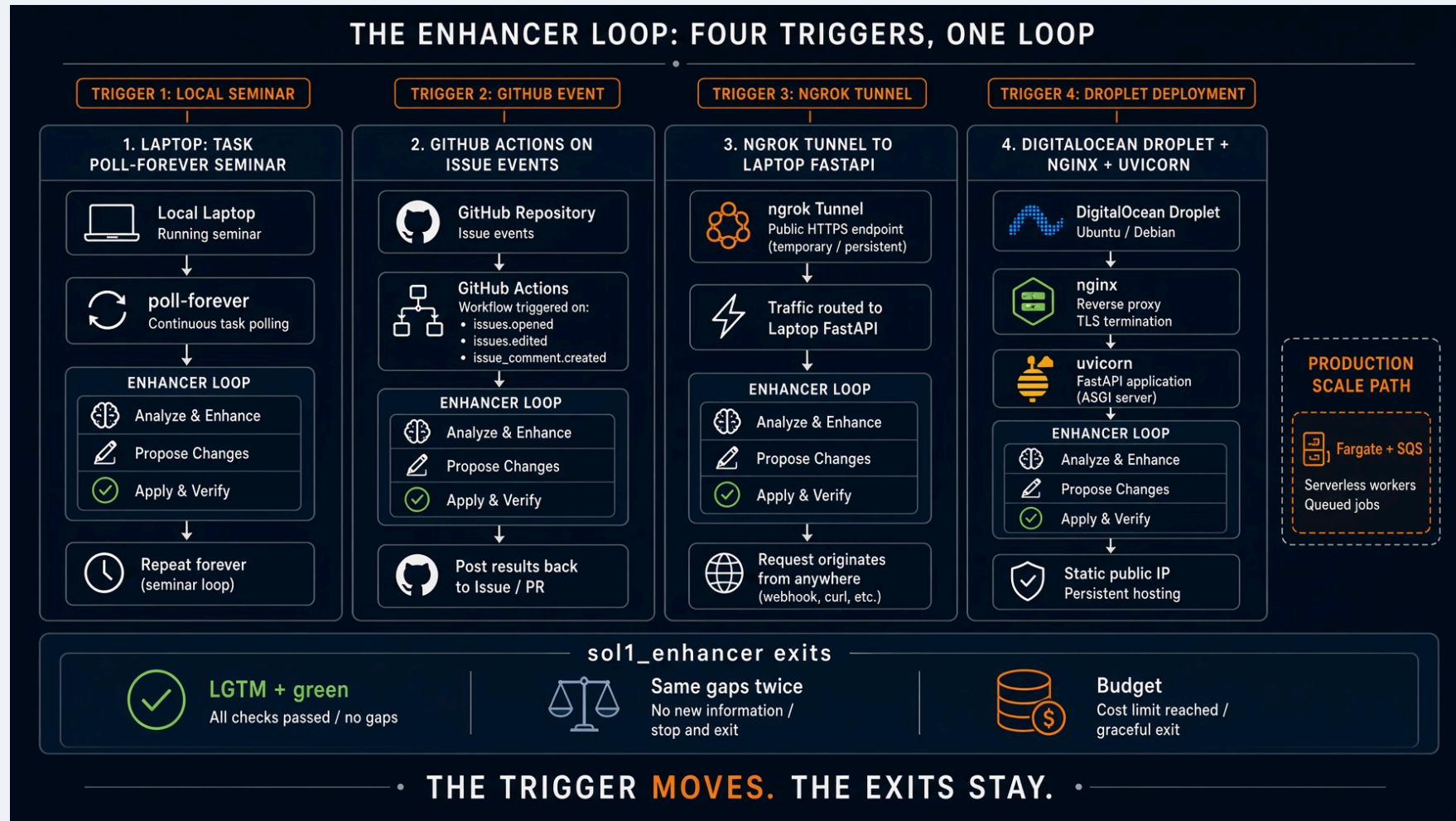
Learning objectives

- Serve `POST /github-webhook` and `GET /health`
- Verify `X-Hub-Signature-256` on the raw body
- Reply 202 before Claude starts
- Lock one issue at a time
- Map `[T001]` in the title to `--ticket T001`
- Skip comments that contain `<!-- enhancer-loop -->`
- Journal every delivery to `work/last-webhook.json`

Starting architecture



The rule on one slide



Same `sol1_enhancer` exits under every trigger:

1. LGTM plus a green rubric
2. Same missing fields twice

Prerequisites

```
cd /path/to/reliable-agentic-lab
python -m uvicorn solutions.extra_credit.s_ext_1_webhook.webhook:app \
  --host 127.0.0.1 --port 8000
curl -s localhost:8000/health
```

Need: FastAPI, uvicorn, `task` on PATH, a filled `solutions/sol1_enhancer`.

The lab stub re-exports that module so extra credit 2 and 5 can keep pointing at `labs/extra-credit/ext_1_webhook/webhook_server.py`.

Step 1. Two routes. Secret first.

Missing secret is 503, not 401. An empty secret is not "unsigned is fine".

```
def secret() -> str:
    return (os.environ.get("GITHUB_WEBHOOK_SECRET") or "").strip()

@app.get("/health")
def health() -> dict:
    return {"ok": True, "backend": backend_name(), "sol1": str(call_sol1.sol1_dir())}
```


Step 2. HMAC on the raw body

```
def verify_signature(body: bytes, header: str | None) -> None:
    value = secret()
    if not value:
        raise HTTPException(status_code=503, detail="GITHUB_WEBHOOK_SECRET is not set")
    if not header:
        raise HTTPException(status_code=401, detail="missing X-Hub-Signature-256")
    digest = hmac.new(value.encode("utf-8"), body, hashlib.sha256).hexdigest()
    expected = "sha256=" + digest
    if not hmac.compare_digest(expected, header.strip()):
        raise HTTPException(status_code=401, detail="bad signature")
```

Never verify against re-serialized JSON. Byte order changes. HMAC then fails for real GitHub too.

Step 3. 202, then work in the background

GitHub waits about 10 seconds. Claude takes minutes.

```
body = await request.body()
verify_signature(body, x_hub_signature_256)
payload = json.loads(body.decode("utf-8") or "{}")
write_journal({..., "status": "accepted"})
background.add_task(handle_delivery, event, payload)
return JSONResponse(preview, status_code=202)
```

A crash in the worker is journaled. It must not become a 500 back to GitHub. GitHub would retry and you would pay twice.

Step 4. Route. Only groom is wired

```
if event == "issues" and action == "opened":  
    return "groom"  
if event == "issue_comment" and action == "created":  
    if "<!-- enhancer-loop -->" in body:  
        return None  
    return "groom"  
if event == "issues" and action == "labeled" and name == "ready":  
    return "fulfill"    # not wired. journal says so.  
if event == "check_suite" and conclusion == "failure":  
    return "fix"        # not wired. Saturday Lab 4.
```

Fulfill and fix stay Saturday labs. Extra credit 1 only starts the enhancer.

Step 5. One lock. Attempt budget. In-progress.

```
if not acquire_issue(number):  
    return {"status": "busy"}  
if attempts >= max_attempts():  
    client.comment(number, f"Giving up after {attempts} attempts ...")  
    return {"status": "gave-up"}  
client.add_label(number, "agent-in-progress")  
try:  
    result = call_sol1.run_sol1(ticket_id)  
finally:  
    client.remove_label(number, "agent-in-progress")
```

AGENT_MAX_ATTEMPTS default 3. Second belt. The loop already has a round budget.

Step 6. Ticket id. Do not invent one.

```
TICKET_IN_TITLE = re.compile(r"\[(T\d+)\]", re.IGNORECASE)
TICKET_IN_BODY  = re.compile(r"(?:^|\n)\s*id:\s*(T\d+)\b", re.IGNORECASE)
```

Title `[T001]` ... is enough. Frontmatter `id: T001` also works. No `id:` comment and stop.

Step 7. Shell out. Never import.

```
BACKEND_FOLDERS = {  
    "claude": "sol1_enhancer",  
    "grok": "sol1_enhancer_grok_build",  
    "opencode": "sol1_enhancer_opencode",  
    "codex": "sol1_enhancer_codex",  
    "agent-sdk": "sol1_enhancer_agent_sdk",  
    "deep-agents": "sol1_enhancer_deep_agents",  
}  
cmd = ["task", "run", "--", "--ticket", ticket_id]  
subprocess.run(cmd, cwd=str(sol1_dir()), timeout=AGENT_TIMEOUT)
```

`AGENT_BACKEND` picks the folder. Default `claude`. Timeout default 900s.

Commands and expected result

```
export GITHUB_WEBHOOK_SECRET=pick-a-long-random-string
python -m uvicorn solutions.extra_credit.s_ext_1_webhook.webhook:app --port 8000
curl -s localhost:8000/health
python -m pytest solutions/extra_credit/s_ext_1_webhook/tests -q
```

Health: {"ok": true, "backend": "claude", "sol1": ".../sol1_enhancer"}.

Tests use `fake_github.py`. No token. The sol1 handoff is monkeypatched.

Troubleshooting

SYMPTOM	CAUSE	FIX
503	secret empty	export <code>GITHUB_WEBHOOK_SECRET</code>
401	secret mismatch, or verified parsed JSON	same string as GitHub. HMAC raw bytes.
202, no Claude	<code>task</code> not on PATH	install Task, check journal <code>stderr</code>
Gave-up comment	<code>AGENT_MAX_ATTEMPTS</code> hit	lower it while testing, or reset labels
Loop replies forever	marker filter missing	skip <code><!-- enhancer-loop --></code>

Validation checklist

- ☐ `/health` names the sol1 folder
- ☐ Unsigned POST is 401
- ☐ Missing secret is 503
- ☐ Signed `issues/opened` is 202
- ☐ Journal has `status: accepted` then a later `ok / sol1-failed`
- ☐ `call_sol1` never `imports` the enhancer package
- ☐ Fulfill and fix routes journal `not-wired`

Recap

What we built. FastAPI that verifies, locks, journals, and starts one poll.

Takeaways

1. Trigger outside. Exits inside.
2. HMAC on the raw body. `compare_digest`.
3. 202 before the model starts.
4. Marker, not author, when reading comments.
5. Next: tunnel it with ngrok, or put it on a Droplet.

Extra credit 2. ngrok adapter

Not Saturday. Run the Lab 1 enhancer on your laptop. GitHub still sends real events.

New trigger. Same exits. Same harness. The graph does not change.

Answer: `solutions/extra_credit/s_ext_2_ngrok/`

No FastAPI. Stdlib `ThreadingHTTPServer`. Port 8765.

Spillwave Solutions | spillwave.com

What you will build

1. Copy the Lab 1 plugin into this folder (`task copy-plugin`)
2. Configure `config.json` and clone your CRM fork
3. Run `bin/webhook_trigger.py`
4. Give GitHub a public URL with ngrok
5. Open an issue titled `[T900] ...` and watch one `task run` start

The adapter is not the loop. It verifies, replies 202, and spawns `task run -- --ticket Txxx`.

Why ngrok

GitHub needs a public HTTPS URL. Your laptop only has localhost.

ngrok opens an outbound tunnel. You do not open a firewall port.

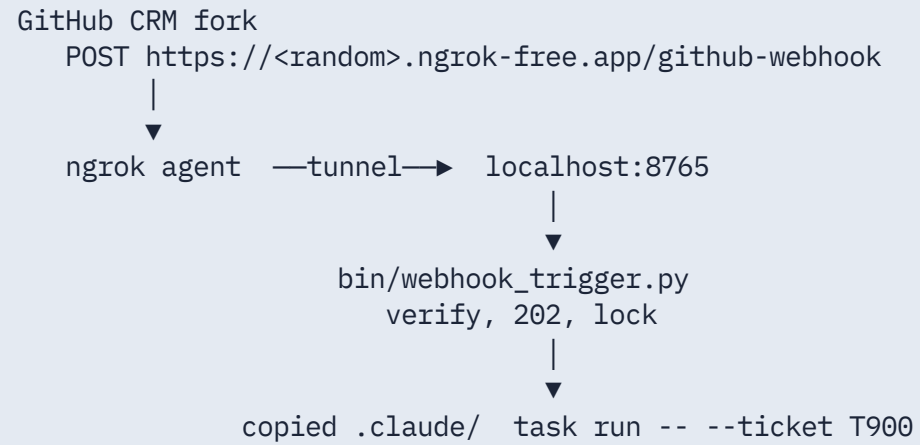
Free tier (August 2026): 3 endpoints, 1 GB, 20k HTTP requests. URLs change on restart. Paid reserved domains stay put.

Learning objectives

- Copy the plugin without importing `sol1_enhancer` at runtime
- Listen on 8765 so the CRM on 8000 is free
- Verify HMAC in Python even if ngrok also checks at the edge
- Dedup on `X-GitHub-Delivery`
- Prove a live GitHub delivery with 202 and `work/last-webhook.json`

Spillwave Solutions | spillwave.com

Starting architecture



Step 1. Copy the plugin here

```
cd solutions/extra_credit/s_ext_2_ngrok  
task copy-plugin
```

Copies `.claude/`, `config.json.example`, `bin/setup_test_tickets.sh`.

Does **not** copy `SPEC.md`, `HOW_TO_RUN.md`, or Lab 1's Taskfile. This folder owns those. Copied `.claude/` is gitignored.

Step 2. Configure and prove one poll by hand

```
cp config.json.example config.json    # set fork_owner
task clone
task create-test-tickets               # optional T900 T901 T902
task run -- --ticket T001
```

That is the exact command the adapter will spawn. If this fails, ngrok will not save you.

Step 3. Three terminals

Terminal A:

```
export GITHUB_WEBHOOK_SECRET='pick-a-long-random-string'  
task listen                # python3 bin/webhook_trigger.py
```

Terminal B:

```
ngrok http 8765
```

Copy the HTTPS URL. Webhook path is `/github-webhook`.

Terminal C:

```
curl -s http://127.0.0.1:8765/health
```

Want `"ok": true` and `"plugin_ready": true`.

Register the webhook on the CRM fork

On **northwind-field-crm**, not this lab repo.

- Payload URL: `https://YOUR-SUBDOMAIN.ngrok-free.app/github-webhook`
- Content type: `application/json`
- Secret: same `GITHUB_WEBHOOK_SECRET`
- Events: Issues, Issue comments. Nothing else.

GitHub sends `ping`. Adapter returns `{"status": "pong"}`. Deliveries tab should show 200.

What the adapter does. Eleven rules.

1. `GET /health` reports whether the copied plugin is present
2. Read the raw body first
3. `HMAC SHA-256 vs X-Hub-Signature-256`. Missing secret 503. Bad sig 401
4. `ping` is pong
5. Title must contain `[Txxx]`
6. `issues` opened, edited, reopened starts one poll
7. `issue_comment` created starts one poll unless the marker is in the body
8. Reply 202 before Claude starts
9. One lock file per ticket under `work/locks/`
10. One file per `X-GitHub-Delivery` under `work/deliveries/`
11. Write `work/last-webhook.json` on every accepted run

Signature and marker. Real code.

```
def verify_signature(payload: bytes, signature: str, secret: str) -> bool:
    if not secret or not signature:
        return False
    if not signature.startswith("sha256="):
        return False
    expected = "sha256=" + hmac.new(secret.encode(), payload, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)
```

```
MARKER = "<!-- enhancer-loop -->"
if MARKER in comment:
    return None
```

Dedup and lock

```
def already_seen(delivery_id: str) -> bool:
    path = work_dir() / "deliveries" / f"{delivery_id}.json"
    if path.exists():
        return True
    path.write_text("{} ", encoding="utf-8")
    return False

def acquire_lock(ticket: str) -> bool:
    fd = os.open(lock_path(ticket), os.O_CREAT | os.O_EXCL | os.O_WRONLY)
```

GitHub retries. At-least-once is the contract. The delivery id file is the idempotency key.

Local HMAC smoke test

```
BODY='{ "action": "opened", "issue": { "number": 1, "title": "[T900] grey button" } }'  
SIG="sha256=$(python3 -c '...hmac...')"  
curl -s -D- -X POST http://127.0.0.1:8765/github-webhook \  
  -H "Content-Type: application/json" \  
  -H "X-GitHub-Event: issues" \  
  -H "X-GitHub-Delivery: local-test-1" \  
  -H "X-Hub-Signature-256: $SIG" \  
  --data "$BODY"
```

Unsigned posts must return 401.

End to end

1. Open [T900] login button is grey on the CRM fork
2. GitHub posts issues/opened. Adapter replies 202
3. task run -- --ticket T900 grooms, posts a marked comment
4. You reply LGTM after the rubric is green
5. Next poll sets ready and loop: implementer

Watch: GitHub deliveries, ngrok inspector `http://127.0.0.1:4040, work/last-webhook.json, work/enhancer-T900.log.`

Troubleshooting

SYMPTOM	CAUSE	FIX
Delivery is HTML	free interstitial	paid domain, or confirm User-Agent is GitHub-Hookshot
401	secret mismatch	same string in GitHub UI and env
503 plugin not copied	<code>.claude/</code> missing	<code>task copy-plugin</code>
202, no Claude	<code>task</code> not on PATH	<code>read work/enhancer-Txxx.log</code>
ngrok URL 404 after restart	free URL rotated	paste the new URL into the webhook
Loop replies forever	missing marker filter	adapter drops the marker

Spillwave Solutions | spillwave.com

Recap

This is still a laptop demo. Close the lid and the tunnel dies.

Production is GitHub Actions, a Droplet, or Fargate. Same one-shot skill. Same state file in `.harness/`.

Spillwave Solutions | spillwave.com

Extra credit 5. DigitalOcean Droplet

Not Saturday. A cheap permanent public endpoint.

New trigger. Same exits. Same harness. The graph does not change.

Cheapest Basic plan. Ubuntu 24.04 LTS. One vCPU. 1 GB RAM. About six dollars a month.

No new Python. You put extra credit 1 on a box.

Spillwave Solutions | spillwave.com

What you will build

The receiver is `solutions/extra_credit/s_ext_1_webhook`. nginx is the only public door. systemd runs uvicorn on loopback.

PIECE	PATH
<code>.env.example</code>	<code>labs/extra-credit/ext_5_digitalocean/.env.example</code>
<code>systemd unit</code>	<code>deploy/agent-webhook.service</code>
<code>nginx</code>	<code>deploy/nginx.conf</code>
<code>sol1 config</code>	<code>deploy/write-sol1-config.sh</code>
<code>handoff</code>	<code>deploy/call-sol1-enhancer.sh T001</code>
<code>smoke</code>	<code>deploy/smoke.sh</code>

Why bind to localhost

The agent holds a token. A default bind of `0.0.0.0` puts it on the public internet.

```
Internet —TLS→ nginx :443 —proxy→ 127.0.0.1:8000 unicorn
```

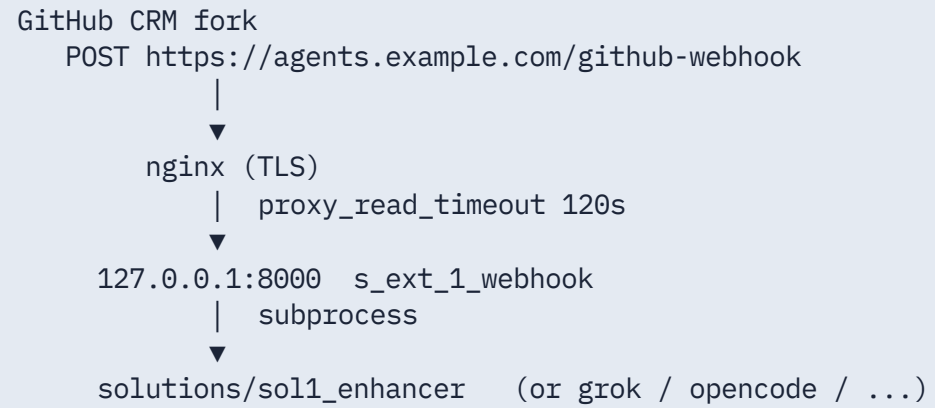
```
ProtectSystem=strict.NoNewPrivileges=yes.User agent.
```

Learning objectives

- Bootstrap Ubuntu 24.04 with one script
- Fill `/opt/agents/.env` from the example
- Terminate TLS with certbot
- Point the **CRM fork** webhook at `https://your-domain/github-webhook`
- Smoke-test HMAC locally before GitHub
- Swap `AGENT_BACKEND` without rewriting the unit

Spillwave Solutions | spillwave.com

Starting architecture



One command on the box

```
ssh root@YOUR_DROPLET_IP
git clone https://github.com/YOUR-FORK/reliable-agentic-lab.git /opt/agents
bash /opt/agents/labs/extra-credit/ext_5_digitalocean/deploy/bootstrap.sh
```

Then edit `/opt/agents/.env`. Point DNS. `certbot --nginx -d your-domain`.

What bootstrap installs

- python3, venv, nginx, certbot, git, jq, task
- system user agent
- clone or pull /opt/agents
- venv at /opt/agent-env with requirements.txt
- .env from the example if missing (chmod 0600)
- optional clone of the CRM fork into work/northwind-field-crm
- write-sol1-config.sh then install.sh

Spillwave Solutions | spillwave.com

`.env` that matters

```
GITHUB_TOKEN=  
GITHUB_REPO=your-github-username/northwind-field-crm  
GITHUB_WEBHOOK_SECRET=  
AGENT_BACKEND=claude  
AGENT_MAX_ATTEMPTS=3  
AGENT_TIMEOUT=900  
WEBHOOK_HOST=127.0.0.1  
WEBHOOK_PORT=8000  
ANTHROPIC_API_KEY=  
WEBHOOK_DOMAIN=agents.example.com
```

`GITHUB_REPO` is the CRM fork, **not** the lab repo.

AGENT_BACKEND picks the folder

```
claude      -> solutions/sol1_enhancer
grok        -> solutions/sol1_enhancer_grok_build
opencode    -> solutions/sol1_enhancer_opencode
codex       -> solutions/sol1_enhancer_codex
agent-sdk   -> solutions/sol1_enhancer_agent_sdk
deep-agents -> solutions/sol1_enhancer_deep_agents
```

Grok is a good fit on a Droplet (you can `task trust` once). It is a poor fit on hosted Actions.

systemd unit. Loopback only.

```
ExecStart=/opt/agent-env/bin/python -m uvicorn \
    solutions.extra_credit.s_ext_1_webhook.webhook:app \
    --host 127.0.0.1 --port 8000
User=agent
EnvironmentFile=/opt/agents/.env
ReadWritePaths=/opt/agents
```

Working directory /opt/agents. PYTHONPATH=/opt/agents.

nginx. Two locations. Everything else 404.

```
location /health          { proxy_pass http://127.0.0.1:8000/health; }
location /github-webhook {
    proxy_pass http://127.0.0.1:8000/github-webhook;
    proxy_read_timeout 120s;
}
location / { return 404; }
```

`client_max_body_size 1m.` Replace `YOUR_DOMAIN` before certbot.

GitHub webhook

On the CRM fork (`GITHUB_REPO`):

- **Payload URL:** `https://your-domain/github-webhook`
- **Content type:** `application/json`
- **Secret:** same as `GITHUB_WEBHOOK_SECRET`
- **Events:** Issues, Issue comments, Check suites

Issue titles must include `[T001]`.

Smoke without GitHub

```
source /opt/agents/.env
bash labs/extra-credit/ext_5_digitalocean/deploy/smoke.sh
sudo -u agent bash -lc 'source /opt/agents/.env && \
/opt/agents/labs/extra-credit/ext_5_digitalocean/deploy/call-sol1-enhancer.sh T001 --print'
```

`smoke.sh` GETs `/health`, then POSTs a signed `issues.opened` body at loopback.

Expected live result

A 202 in the GitHub webhook log. A record in:

```
/opt/agents/solutions/extra_credit/s_ext_1_webhook/work/last-webhook.json
```

A task run in `solutions/sol1_enhancer` for that ticket.

```
journalctl -u agent-webhook -f
```


Troubleshooting

SYMPTOM	CAUSE	FIX
Droplet 502	uvicorn down, or proxy_pass wrong	<code>systemctl status agent-webhook</code>
401	secret mismatch	same string in GitHub and <code>.env</code>
503	secret empty	fill <code>.env</code> , restart unit
Clone failed	<code>CRM_OWNER</code> still the placeholder	<code>edit .env</code> , run <code>write-sol1-config.sh</code>
Grok skill not found	untrusted git root	<code>task trust</code> as the agent user

Recap

Permanent HTTPS. Same receiver as extra credit 1. Same exits as Lab 1.

The lid can close. The box keeps listening.

Next scale step: Fargate plus a queue. Same HMAC. Same 202. Worker is a second service.

Deploy on GitHub Actions

Ticket change events. One poll. Then exit.

New trigger. Same exits. Same harness. The graph does not change.

Copy-me workflow: `labs/lab1_enhancer/workflows/enhance-on-issue.yml`

Notes: `labs/lab1_enhancer/GITHUB-ACTIONS.md`

Not Saturday. Copy onto **your CRM fork**. Never the instructor repo.

Spillwave Solutions | spillwave.com

What you will deploy

A workflow that GitHub already knows how to start.

```
on:
  issues:
    types: [opened, edited, labeled]
  issue_comment:
    types: [created]
  workflow_dispatch:
```

That is "the ticket changed": a new issue, an edited body, a label, or a human comment. LGTM arrives as `issue_comment`.

Why Actions instead of polling

`task poll-forever` dies when the laptop sleeps.

GitHub already emitted the event. Hosting a receiver is optional.

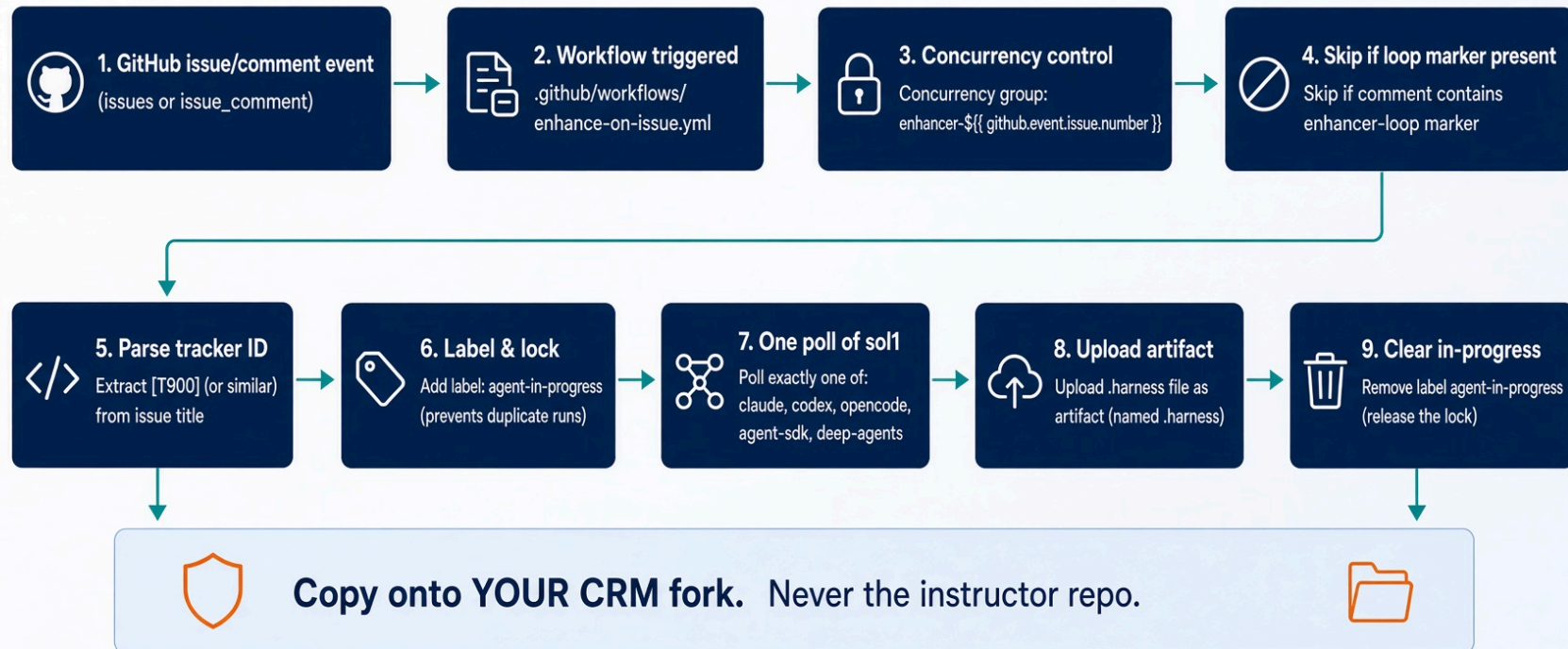
The trigger starts one poll. The enhancer exits. The workflow does not grade the ticket, merge, or deploy.

Learning objectives

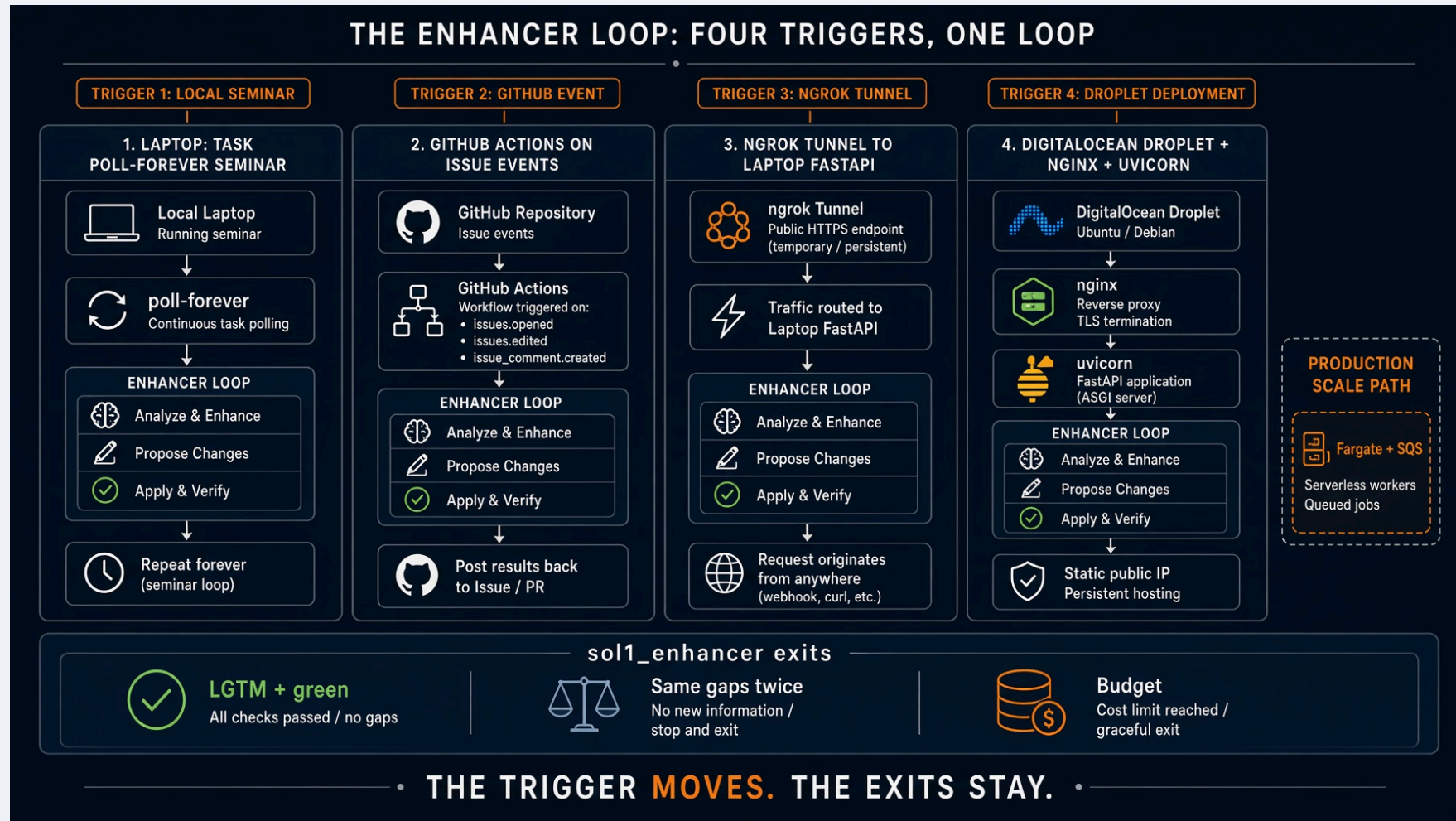
- Copy the YAML onto the CRM fork
- Parse `[T900]` from the issue title
- Skip `<!-- enhancer-loop -->`
- Serialize with `concurrency.group` per issue
- Swap `ENHANCER_BACKEND` without rewriting jobs
- Know which backends do **not** belong on hosted runners

Starting architecture

GitHub Actions as the receiver



Four triggers. Same exits.



Actions is column two. The loop is still `sol1_*`.

Copy onto YOUR fork

```
mkdir -p .github/workflows  
cp path/to/reliable-agentlab/labs/lab1_enhancer/workflows/enhance-on-issue.yml \  
  .github/workflows/enhance-on-issue.yml
```

Do not enable this on the shared instructor repo. It comments on issues and can spend model tokens.

Permissions. Least you need.

```
permissions:  
  issues: write  
  contents: read  
  actions: read
```

Grant `contents: write` only if a later job will push. The enhancer edits issue bodies and labels, not app code.

A workflow that merges has left the lab.

Concurrency per issue

```
concurrency:  
  group: enhancer-${ github.event.issue.number || github.event.inputs.ticket }  
  cancel-in-progress: false
```

Two events on the same ticket must not chew it twice. Do not cancel in progress. A mid-poll kill leaves labels and a half-written candidate.

Skip the loop's own comments

```
if: >
  github.event_name == 'workflow_dispatch' ||
  github.event_name == 'issues' ||
  (github.event_name == 'issue_comment' &&
    !contains(github.event.comment.body, '<!-- enhancer-loop -->'))
```

Filter on the marker, never the author. The job runs as `github-actions[bot]`. A human LGTM must still count.

Ticket id from the title

```
id="$(printf '%s\n' "$TITLE" | grep -oE '\[T[0-9]+\]' | head -1 | tr -d '[]')"
if [ -z "$id" ]; then
    id="T${NUMBER}"
fi
```

Seed titles look like `[T900] Search crashes on an empty query`. If the title has no id, `T{issue_number}` matches `HOW_TO_RUN` for issues opened in the UI.

Backend matrix

ENHANCER_BACKEND	FOLDER	HOW THE JOB STARTS IT
claude (default)	sol1_enhancer	task run -- --ticket "\$TICKET"
codex	sol1_enhancer_codex	same
opencode	sol1_enhancer_opencode	same
agent-sdk	sol1_enhancer_agent_sdk	python3 loop.py --once ...
deep-agents	sol1_enhancer_deep_agents	python3 loop.py --once ...

Set repository variable ENHANCER_BACKEND. Secret ANTHROPIC_API_KEY for SDK and Deep Agents.

What does not belong on hosted runners

Grok Build. Trust prompt. Local plugin shims. `task trust` is a laptop step.

Keep Grok on a Droplet (`ext_5`) or a laptop (`ext_2`).

Codex needs `codex` on the runner. If the image does not have it, use Agent SDK or Claude Code.

In-progress and the second belt

```
gh issue edit "$NUMBER" --add-label agent-in-progress  
# ... one poll ...  
gh issue edit "$NUMBER" --remove-label agent-in-progress # if: always()
```

Stop at `AGENT_MAX_ATTEMPTS` (default 3) by counting `agent-attempts-*` labels. The loop already has a round budget. This is a second belt, at the trigger.

Two checkouts

```
- uses: actions/checkout@v4          # the CRM fork, $GITHUB_WORKSPACE
- uses: actions/checkout@v4
  with:
    repository: ${vars.ENHANCER_REPO || github.repository}
    path: .enhancer
```

`--repo "$GITHUB_WORKSPACE"` points the enhancer at the CRM files. The enhancer code lives in `.enhancer`.

Run one poll

```
case "$ENHANCER_BACKEND" in
  claude)    cd solutions/sol1_enhancer && task run -- --ticket "$TICKET" --repo "$TARGET" ;;
  agent-sdk) cd solutions/sol1_enhancer_agent_sdk && python3 loop.py --once --repo "$TARGET" --ticket "$TICKET" ;;
  deep-agents) cd solutions/sol1_enhancer_deep_agents && python3 loop.py --once --repo "$TARGET" --ticket "$TICKET" ;;
esac
```

Upload `.harness/last-enhancer-*.json` as an artifact. That is how you read the last score at 2 a.m.

What does not change

Three exits, no fourth: LGTM plus a green rubric, same gaps twice, budget spent.

Labels enhanced, ready, needs-human. Judge still has no write tool. `check_fields.py` still computes ready.

Verify

1. Push the workflow to **your** fork
2. Open [T900] Search crashes on an empty query
3. Actions tab shows enhance. Issue gets enhanced or a marked comment
4. Comment LGTM as yourself. Next event sets ready only if the rubric is already green
5. A second event with no new human comment must be a no-op. If it posts again, the marker filter is missing

Troubleshooting

SYMPTOM	CAUSE	FIX
Never fires	workflow on the instructor repo	copy YAML to your fork
Loops	marker filter missing	skip <code><!-- enhancer-loop --></code>
404 on issues	token cannot write issues	issues: <code>write</code> , or a PAT for a private CRM
Grok skill not found	hosted runner	use <code>claude / agent-sdk / deep-agents</code>
No artifact	harness path wrong	upload <code>**/.harness/last-enhancer-*.json</code>

When Actions is the wrong tool

Laptop demo: extra credit 2, ngrok.

Permanent cheap box: extra credit 5, Droplet.

Long-running model calls, private VPC, or you already live on AWS: Fargate. Next deck.

If you stall, return to `task run --from labs/lab1_enhancer`. That is the Saturday path.

Spillwave Solutions | spillwave.com

Recap

Takeaways

1. Copy onto the CRM fork.
2. Trigger starts. Loop stops.
3. Marker, not author.
4. Concurrency per issue. Never cancel mid-poll.
5. Grok stays off hosted runners.

Closing line. The workflow is a doorbell. It is not the loop.

Spillwave Solutions | spillwave.com

Deploy on AWS Fargate

Production pattern. **Not a shipped lab.** There is no `ext_fargate` folder.

Reuses extra credit 1: verify HMAC, reply 202, then run `sol1_enhancer`.

The new knob is a queue. GitHub's 10-second budget cannot wait for Claude.

Spillwave Solutions | spillwave.com

What you will build (architecture, not a drop-in)

PIECE	JOB
ALB	public HTTPS, GitHub only
Fargate service <code>webhook-receiver</code>	verify, dedup, enqueue, 202
SQS	at-least-once buffer
Fargate service <code>enhancer-worker</code>	<code>task run -- --ticket Txxx</code>
Secrets Manager	webhook secret, <code>GITHUB_TOKEN</code> , model keys
DynamoDB	delivery-id idempotency
CloudWatch	journal you can read at 2 a.m.

Same exits as Lab 1. The trigger moved again.

New trigger. Same exits. Same harness. Fargate splits receive from work. That is still not a new loop.

Why Fargate, not Lambda

Apoll can run 15 minutes (`AGENT_TIMEOUT=900`). Lambda's practical ceiling is a poor fit once the model is in the loop.

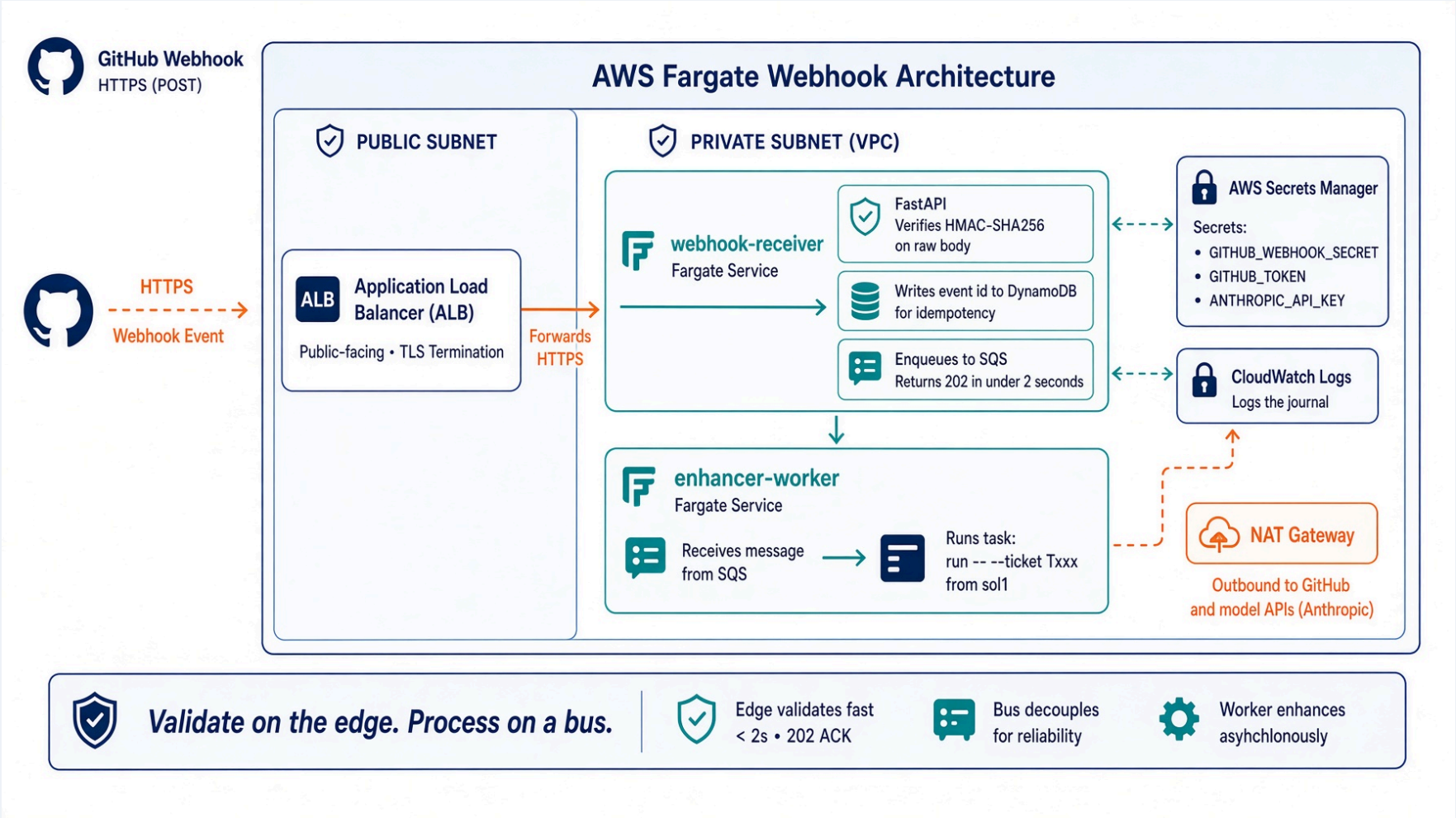
Fargate task CPU and memory are sized for the **worker**, not the receiver. The receiver should be tiny. The worker should be the size of Claude plus git.

GitHub webhooks: return 2xx fast. Process on a bus. (2026 webhook practice.)

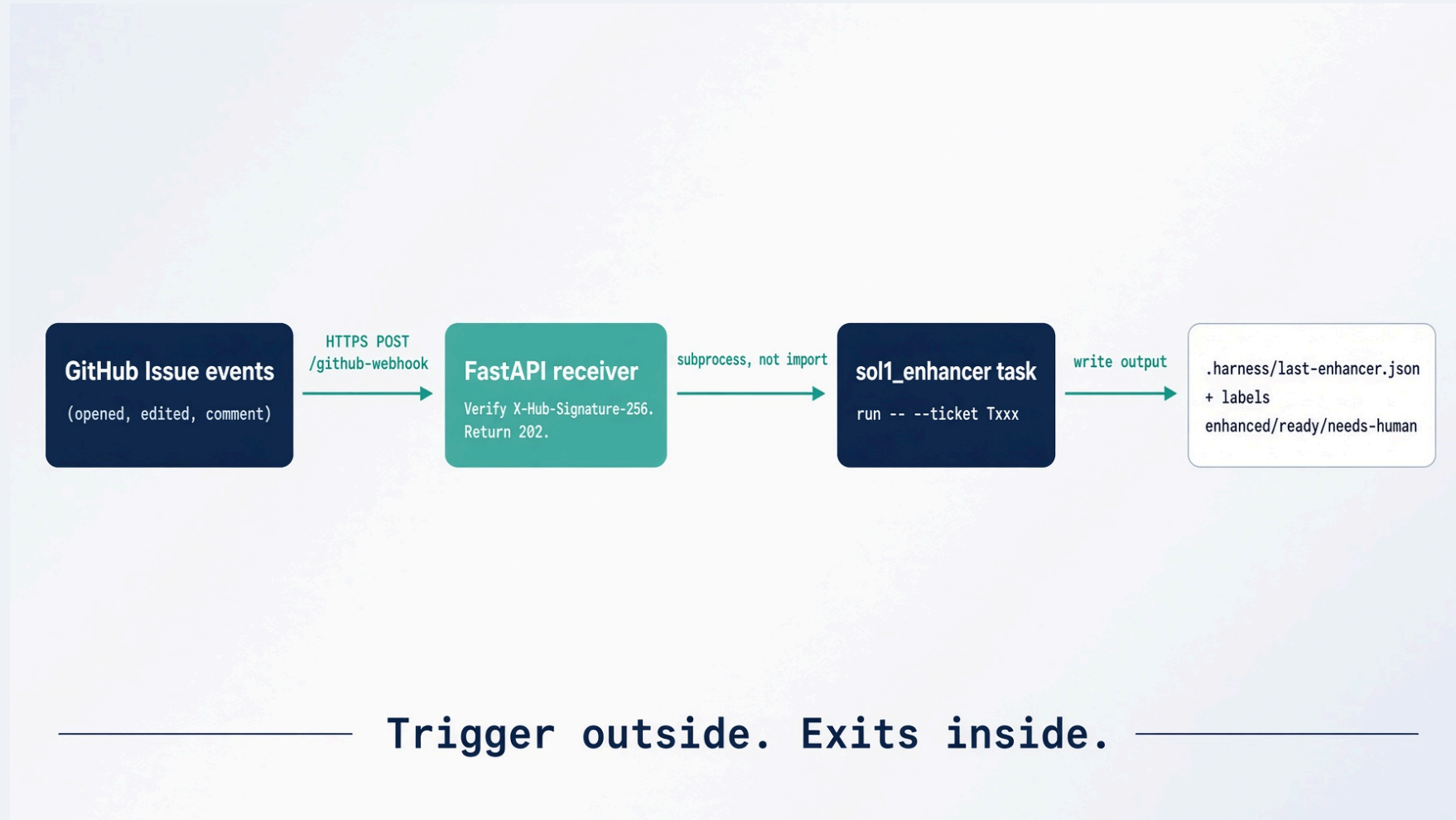
Learning objectives

- Split receive from work
- Verify HMAC on the raw ALB body
- Persist `X-GitHub-Delivery` before enqueue
- Keep the receiver on a private subnet behind ALB
- Inject secrets. Never bake tokens in the image
- Map this pattern back to `s_ext_1_webhook.webhook.py`

Starting architecture



Same rule as extra credit



On Fargate the teal box becomes "verify + SQS". The navy task run box becomes the worker service.

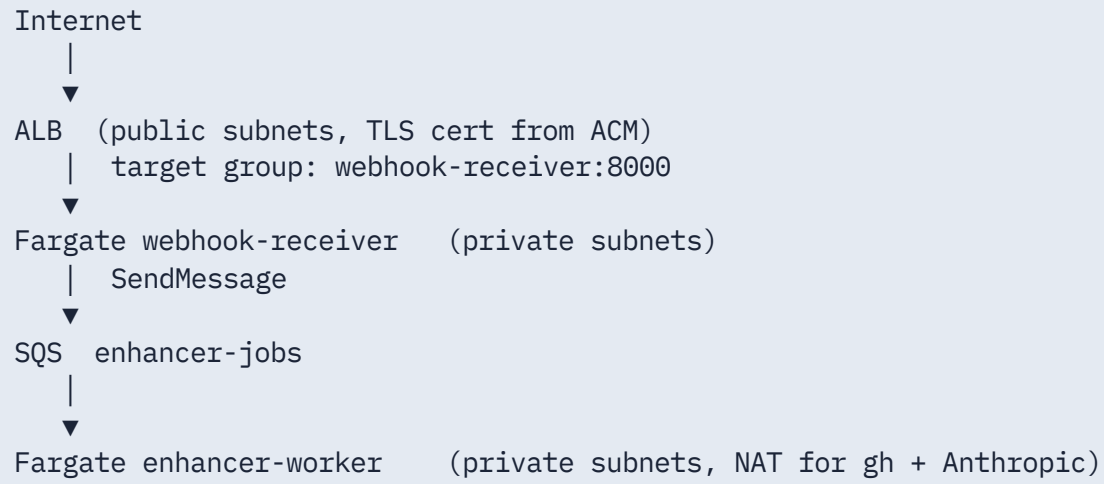
Assumption. Say it out loud.

This deck is a production mapping of extra credit 1. The repo does **not** ship Terraform.

If you implement it, keep:

- `verify_signature` byte-for-byte from `webhook.py`
- `202` before work
- marker skip
- `AGENT_MAX_ATTEMPTS`
- subprocess, not import
- no merge tool

Network. Private tasks. Public ALB.



Assign public IPs only if you skip NAT. Do not. The worker holds a token.

Receiver. Keep extra credit 1's contract.

```
body = await request.body()
verify_signature(body, x_hub_signature_256)    # 401 / 503
if already_seen(x_github_delivery):            # DynamoDB Put with cond
    return JSONResponse({"status": "duplicate"}, 200)
sqs.send_message(QueueUrl=..., MessageBody=body.decode(),
                 MessageAttributes={"event": x_github_event})
return JSONResponse({"status": "accepted"}, 202)
```

Target p95 under 2 seconds. Alert if it creeps toward GitHub's 10-second budget.

Idempotency. Delivery id is the key.

GitHub retries. SQS is at-least-once. Two belts:

1. DynamoDB item `delivery_id` with a condition `attribute_not_exists`
2. Worker lock per ticket (the extra-credit `LOCK_DIR` idea, now a DynamoDB lock or SQS message group id)

FIFO SQS with `MessageGroupId = issue_number` serializes one ticket. Standard SQS plus a lock is the other shape. Pick one.

Worker. Same subprocess as the Droplet.

```
# enhancer-worker: long-running consumer
ticket = ticket_id_from_issue(payload)
run_sol1(ticket)    # call_sol1.py, AGENT_BACKEND, timeout 900
```

Image contains `task`, the lab checkout, and the chosen backend CLI. Task definition `ulimits` and ephemeral storage must fit a git clone of the CRM.

Scale the worker on SQS depth, not on ALB request count.

Secrets. Three, maybe four.

SECRET	WHO READS IT
GITHUB_WEBHOOK_SECRET	receiver only
GITHUB_TOKEN	worker (labels, comments, issue body)
ANTHROPIC_API_KEY	worker, when backend is claude / agent-sdk / deep-agents
OPENAI_API_KEY / XAI_API_KEY	worker, matching AGENT_BACKEND

Task definition: secrets from Secrets Manager. Rotate the webhook secret on a 90-day calendar. Accept both during the window.

IAM. Two roles, not one.

Receiver task role: `sqs:SendMessage`, `dynamodb:PutItem`, `secretsmanager:GetSecretValue` for the webhook secret only.

Worker task role: `sqs:ReceiveMessage/DeleteMessage`, `dynamodb:Get/Put` for locks, `secretsmanager` for token and model keys. No `sqs:SendMessage` on this queue.

Execution role: pull from ECR, write CloudWatch. That is not the task role.

Health, timeouts, drain

ALB health check: `GET /health` on the receiver. Same payload as extra credit 1.

Receiver `stopTimeout` short. Worker `stopTimeout` long enough to finish one poll or nack the SQS message.

`proxy_read_timeout` on the Droplet was 120s. Here the ALB idle timeout can stay at 60s because work is not on that socket.

Backend choices on Fargate

BACKEND	ON FARGATE?
agent-sdk / deep-agents	best fit. Python in the image, key in Secrets Manager
claude CLI	works if you install it at build. Headless -p
opencode	works if the image has it
codex	possible. Watch the sandbox flags from bin/role.sh
grok	poor fit. Trust prompt, local shims. Keep on a Droplet

Spillwave Solutions | spillwave.com

Observability. The 2 a.m. test.

Ship `last-webhook.json` fields to CloudWatch as structured logs: delivery, issue, ticket, backend, returncode.

Alarm on:

- receiver 4xx rate (bad signatures)
- receiver p95 latency
- SQS age of oldest message
- worker non-zero returncode
- gave-up comments

If you cannot read the last score, you cannot debug unattended.

Security extras GitHub already gives you

Allow the ALB security group from GitHub webhook IP ranges, or put a WAF with a GitHub Hookshot user-agent plus HMAC (HMAC is the real gate). IP allowlists drift. HMAC does not.

Reject bodies over 1 MB, same as the nginx `client_max_body_size`.

Never log the raw secret or the `Authorization` header.

Walkthrough. Twelve steps.

1. ECR repo, image from extra credit 1 plus `call_sol1.py`
2. VPC: 2 public, 2 private, NAT
3. ACM cert on the hostname GitHub will call
4. ALB + target group + HTTPS listener
5. SQS queue (FIFO if you want per-issue serialization)
6. DynamoDB table `deliveries` (pk: `delivery_id`)
7. Secrets Manager entries
8. Receiver service, desired count 2
9. Worker service, desired count 1, scale on queue depth
10. Point the CRM fork webhook at `https://agents.example.com/github-webhook`
11. Open `[T900]` Watch 202, then a worker log, then a marked comment
12. Comment `LGTM` on a green rubric. Next message should set `ready`

Troubleshooting

SYMPTOM	CAUSE	FIX
GitHub delivery timeout	work on the request thread	enqueue, then 202
401 from ALB	body decoded then re-encoded	HMAC the raw bytes ALB gave you
Duplicate polls	no delivery-id table	DynamoDB condition put
Worker cannot gh	no NAT, or token missing	private subnet plus NAT. Secret on the worker
Scale to zero, cold 15s	receiver scaled to 0	keep desired count >= 1, or accept the miss

Recap

What changed. The doorbell is an ALB. The waiting room is SQS. The loop is still `soll_*`.

What did not change. HMAC. 202. Marker. Three exits. No merge.

Takeaways

1. Validate on the edge. Process on a bus.
2. Size the worker for the model, the receiver for HMAC.
3. Two IAM roles.
4. Grok still belongs on a Droplet.
5. If you cannot read the last score, you cannot debug at 2 a.m.

Closing line. Fargate is a bigger doorbell. It is still not the loop.